

每次调用 open 时都会确定文件模式

对于一个给定流，每当打开文件时，都可以改变其文件模式。

```
ofstream out; // 未指定文件打开模式
out.open("scratchpad"); // 模式隐含设置为输出和截断
out.close(); // 关闭 out，以便我们将其用于其他文件
out.open("precious", ofstream::app); // 模式为输出和追加
out.close();
```

第一个 open 调用未显式指定输出模式，文件隐式地以 out 模式打开。通常情况下，out 模式意味着同时使用 trunc 模式。因此，当前目录下名为 scratchpad 的文件的内容将被清空。当打开名为 precious 的文件时，我们指定了 append 模式。文件中已有的数据都得以保留，所有写操作都在文件末尾进行。

在每次打开文件时，都要设置文件模式，可能是显式地设置，也可能是隐式地设置。当程序未指定模式时，就使用默认值。

8.2.2 节练习

练习 8.7: 修改上一节的书店程序，将结果保存到一个文件中。将输出文件名作为第二个参数传递给 main 函数。

练习 8.8: 修改上一题的程序，将结果追加到给定的文件末尾。对同一个输出文件，运行程序至少两次，检验数据是否得以保留。

8.3 string 流

321

sstream 头文件定义了三个类型来支持内存 IO，这些类型可以向 string 写入数据，从 string 读取数据，就像 string 是一个 IO 流一样。

istringstream 从 string 读取数据，ostreamstream 向 string 写入数据，而头文件 stringstream 既可从 string 读数据也可向 string 写数据。与 fstream 类型类似，头文件 sstream 中定义的类型都继承自我们已经使用过的 iostream 头文件中定义的类型。除了继承得来的操作，sstream 中定义的类型还增加了一些成员来管理与流相关联的 string。表 8.5 列出了这些操作，可以对 stringstream 对象调用这些操作，但不能对其他 IO 类型调用这些操作。

表 8.5: stringstream 特有的操作	
sstream strm;	strm 是一个未绑定的 stringstream 对象。sstream 是头文件 sstream 中定义的一个类型
sstream strm(s);	strm 是一个 sstream 对象，保存 strings 的一个拷贝。此构造函数是 explicit 的（参见 7.5.4 节，第 265 页）
strm.str()	返回 strm 所保存的 string 的拷贝
strm.str(s)	将 string s 拷贝到 strm 中。返回 void

8.3.1 使用 istringstream

当我们的某些工作是对整行文本进行处理，而其他一些工作是处理行内的单个单词

时,通常可以使用 `istringstream`。

考虑这样一个例子,假定有一个文件,列出了一些人和他们的电话号码。某些人只有一个号码,而另一些人则有多个——家庭电话、工作电话、移动电话等。我们的输入文件看起来可能是这样的:

```
morgan 2015552368 8625550123
drew 9735550130
lee 6095550132 2015550175 8005550000
```

文件中每条记录都以一个人名开始,后面跟随一个或多个电话号码。我们首先定义一个简单的类来描述输入数据:

```
// 成员默认为公有;参见 7.2 节(第 240 页)
struct PersonInfo {
    string name;
    vector<string> phones;
};
```

类型 `PersonInfo` 的对象会有一个成员来表示人名,还有一个 `vector` 来保存此人的所有电话号码。

322

我们的程序会读取数据文件,并创建一个 `PersonInfo` 的 `vector`。`vector` 中每个元素对应文件中的一条记录。我们可以在一个循环中处理输入数据,每个循环步读取一条记录,提取出一个人名和若干电话号码:

```
string line, word; // 分别保存来自输入的一行和单词
vector<PersonInfo> people; // 保存来自输入的所有记录
// 逐行从输入读取数据,直至 cin 遇到文件尾(或其他错误)
while (getline(cin, line)) {
    PersonInfo info; // 创建一个保存此记录数据的对象
    istringstream record(line); // 将记录绑定到刚读入的行
    record >> info.name; // 读取名字
    while (record >> word) // 读取电话号码
        info.phones.push_back(word); // 保持它们
    people.push_back(info); // 将此记录追加到 people 末尾
}
```

这里我们用 `getline` 从标准输入读取整条记录。如果 `getline` 调用成功,那么 `line` 中将保存着从输入文件而来的一条记录。在 `while` 中,我们定义了一个局部 `PersonInfo` 对象,来保存当前记录中的数据。

接下来我们将一个 `istringstream` 与刚刚读取的文本行进行绑定,这样就可以在此 `istringstream` 上使用输入运算符来读取当前记录中的每个元素。我们首先读取人名,随后用一个 `while` 循环读取此人的电话号码。

当读取完 `line` 中所有数据后,内层 `while` 循环就结束了。此循环的工作方式与前面章节中读取 `cin` 的循环很相似,不同之处是,此循环从一个 `string` 而不是标准输入读取数据。当 `string` 中的数据全部读出后,同样会触发“文件结束”信号,在 `record` 上的下一个输入操作会失败。

我们将刚刚处理好的 `PersonInfo` 追加到 `vector` 中,外层 `while` 循环的一个循环步就随之结束了。外层 `while` 循环会持续执行,直至遇到 `cin` 的文件结束标识。

8.3.1 节练习

练习 8.9: 使用你为 8.1.2 节 (第 281 页) 第一个练习所编写的函数打印一个 `istreamstream` 对象的内容。

练习 8.10: 编写程序, 将来自一个文件中的行保存在一个 `vector<string>` 中。然后使用一个 `istreamstream` 从 `vector` 读取数据元素, 每次读取一个单词。

练习 8.11: 本节的程序在外层 `while` 循环中定义了 `istreamstream` 对象。如果 `record` 对象定义在循环之外, 你需要对程序进行怎样的修改? 重写程序, 将 `record` 的定义移到 `while` 循环之外, 验证你设想的修改方法是否正确。

练习 8.12: 我们为什么没有在 `PersonInfo` 中使用类内初始化?

8.3.2 使用 `ostreamstream`

323

当我们逐步构造输出, 希望最后一起打印时, `ostreamstream` 是很有用的。例如, 对上一节的例子, 我们可能想逐个验证电话号码并改变其格式。如果所有号码都是有效的, 我们希望输出一个新的文件, 包含改变格式后的号码。对于那些无效的号码, 我们不会将它们输出到新文件中, 而是打印一条包含人名和无效号码的错误信息。

由于我们不希望输出有无效电话号码的人, 因此对每个人, 直到验证完所有电话号码后才可以进行输出操作。但是, 我们可以先将输出内容“写入”到一个内存 `ostreamstream` 中:

```
for (const auto &entry : people) { // 对 people 中每一项
    ostreamstream formatted, badNums; // 每个循环步创建的对象
    for (const auto &nums : entry.phones) { // 对每个数
        if (!valid(nums)) {
            badNums << " " << nums; // 将数的字符串形式存入 badNums
        } else
            // 将格式化的字符串“写入” formatted
            formatted << " " << format(nums);
    }
    if (badNums.str().empty()) // 没有错误的数
        os << entry.name << " " // 打印名字
        << formatted.str() << endl; // 和格式化的数
    else // 否则, 打印名字和错误的数
        cerr << "input error: " << entry.name
        << " invalid number(s) " << badNums.str() << endl;
}
```

在此程序中, 我们假定已有两个函数, `valid` 和 `format`, 分别完成电话号码验证和改变格式的功能。程序最有趣的部分是对字符串流 `formatted` 和 `badNums` 的使用。我们使用标准的输出运算符(`<<`)向这些对象写入数据, 但这些“写入”操作实际上转换为 `string` 操作, 分别向 `formatted` 和 `badNums` 中的 `string` 对象添加字符。

8.3.2 节练习

练习 8.13: 重写本节的电话号码程序, 从一个命名文件而非 `cin` 读取数据。

练习 8.14: 我们为什么将 `entry` 和 `nums` 定义为 `const auto&`?

小结

C++使用标准库类来处理面向流的输入和输出:

- `iostream` 处理控制台 IO
- `fstream` 处理命名文件 IO
- `stringstream` 完成内存 `string` 的 IO

类 `fstream` 和 `stringstream` 都是继承自类 `iostream` 的。输入类都继承自 `istream`, 输出类都继承自 `ostream`。因此, 可以在 `istream` 对象上执行的操作, 也可在 `ifstream` 或 `istringstream` 对象上执行。继承自 `ostream` 的输出类也有类似情况。

每个 IO 对象都维护一组条件状态, 用来指出此对象上是否可以进行 IO 操作。如果遇到了错误——例如在输入流上遇到了文件末尾, 则对象的状态变为失效, 所有后续输入操作都不能执行, 直至错误被纠正。标准库提供了一组函数, 用来设置和检测这些状态。

术语表

条件状态 (condition state) 可被任何流类使用的一组标志和函数, 用来指出给定流是否可用。

文件模式 (file mode) 类 `fstream` 定义的一组标志, 在打开文件时指定, 用来控制文件如何被使用。

文件流 (file stream) 用来读写命名文件的流对象。除了普通的 `iostream` 操作, 文件流还定义了 `open` 和 `close` 成员。成员函数 `open` 接受一个 `string` 或一个 C 风格字符串参数, 指定要打开的文件名, 它还可以接受一个可选的参数, 指明文件打开模式。成员函数 `close` 关闭流所关联的文件, 调用 `close` 后才可以调用 `open` 打开另一个文件。

`fstream` 用于同时读写一个相同文件的文件流。默认情况下, `fstream` 以 `in` 和 `out` 模式打开文件。

`ifstream` 用于从输入文件读取数据的文件流。默认情况下, `ifstream` 以 `in` 模式打开文件。

继承 (inheritance) 程序设计功能, 令一个类型可以从另一个类型继承接口。类 `ifstream` 和 `istringstream` 继承自 `istream`, `ofstream` 和 `ostringstream` 继承自 `ostream`。第 15 章将介绍继承。

`istringstream` 用来从给定 `string` 读取数据的字符串流。

`ofstream` 用来向输出文件写入数据的文件流。默认情况下, `ofstream` 以 `out` 模式打开文件。

字符串流 (string stream) 用于读写 `string` 的流对象。除了普通的 `iostream` 操作外, 字符串流还定义了一个名为 `str` 的重载成员。调用 `str` 的无参版本会返回字符串流关联的 `string`。调用时传递给它一个 `string` 参数, 则会将字符串流与该 `string` 的一个拷贝相关联。

`stringstream` 用于读写给定 `string` 的字符串流。